Operating Systems and System Programming

Fall 2025

Chapter II: Process Management

II.1. Overview

In operating systems, processes and threads are the fundamental units of execution.

This chapter uncovers their structure, behavior, and management to understand how multitasking and concurrency work.

Outline

- What is a Process?
- What is a Thread?
- Processes vs Threads
- Implementation of Processes and Threads
- Conclusion

Outline

- **What is a Process?**
- What is a Thread?
- Processes vs Threads
- Implementation of Processes and Threads
- Conclusion

II.2. What is a Process?

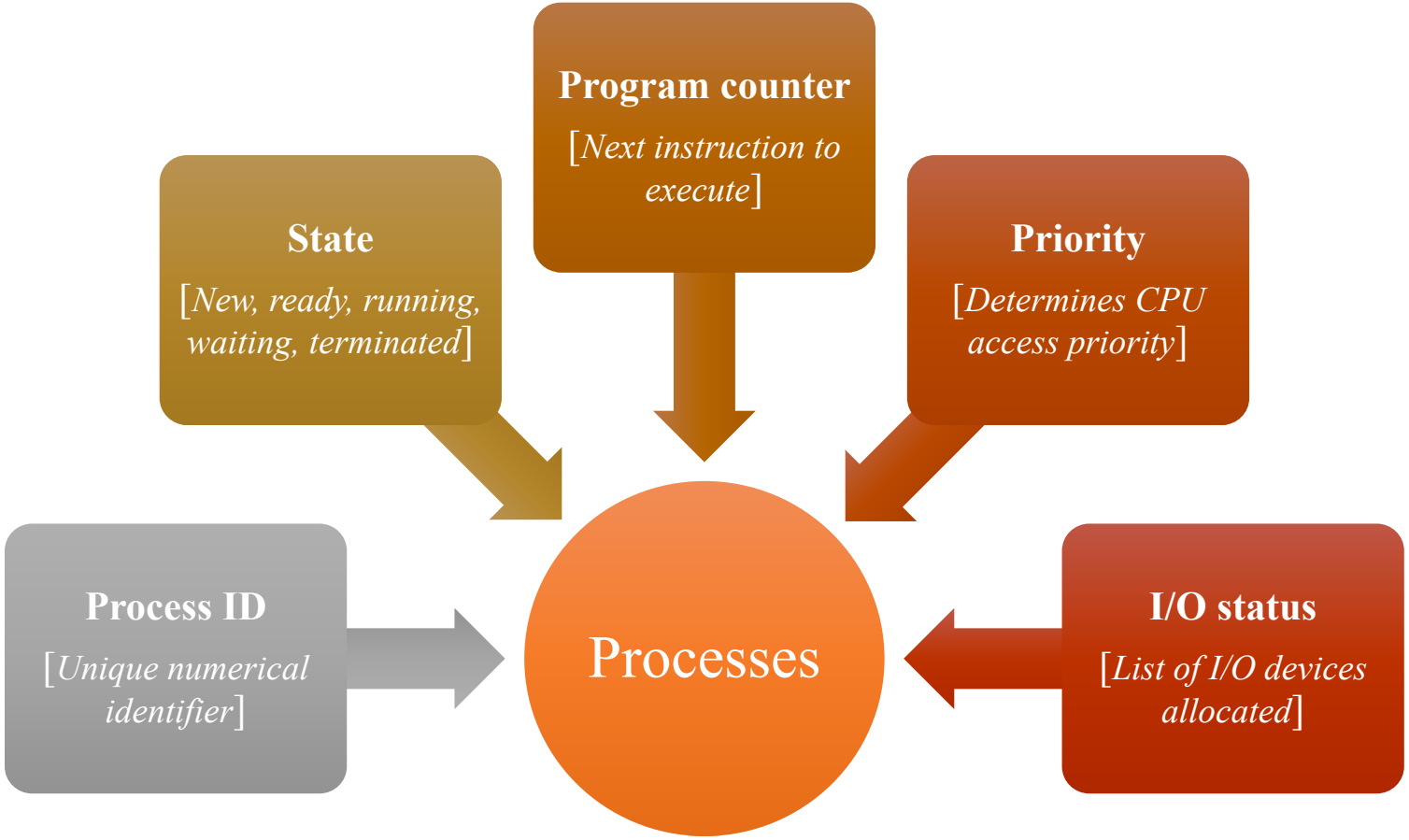
A process is **an instance of a program in execution**, consisting of:

- Program code (text section),
- Current activities (program counter, registers),
- Associated resources (memory, files, CPU state).

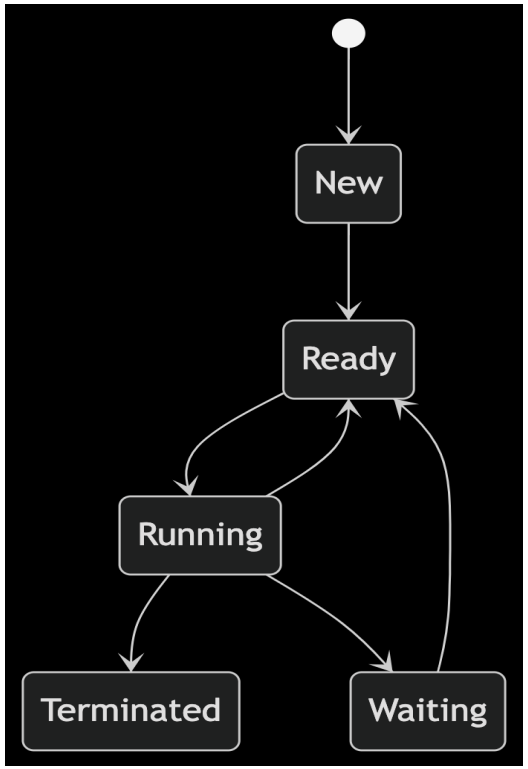
It is characterized by:

- Isolation: Each process has its own memory space
- Heavyweight: Requires more system resources
- Context switching: slower due to memory isolation
- Communication: Needs inter-process communication (IPC) mechanism.

Processes have:



States



II.2.1. Type of Process

Processes can be categorized as follows:

Type	Description
Foreground	Interacts directly with the user
Background	Runs without user interaction
Daemon	Background service that starts at boot
Parent	Process that creates other processes
Child	Process created by another process
Zombie	A terminated process that still has an entry in process table
Orphan	A child process whose parent has terminated

Outline

- What is a Process?
- **What is a Thread?**
- Processes vs Threads
- Implementation of Processes and Threads
- Conclusion

II.3. What is a Thread?

A thread is **a smallest unit in execution within a process**, sharing the process's memory and resources.

It is characterized by:

- ❑ Shared memory: threads within the same process share memory,
- ❑ Lightweight: Less overhead than process,
- ❑ Faster context switching: Due to shared memory,
- ❑ Simplified Communication: No need for IPC mechanism.

II.3.1. Type of Threads

Threads can be categorized as follows:

Type	Description	Example
User-level	Managed by user-level libraries; fast but not visible to OS	Video game engine threads managing rendering. If it blocks, the game can bug
Kernel-level	Managed directly by the OS, slower but powerful	Multiple client requests simultaneously managed via a web server. If one block, other can still continue
Hybrid	Combines user and kernel-level management	Used by Solaris OS

Outline

- What is a Process?
- What is a Thread?
- **Processes vs Threads**
- Implementation of Processes and Threads
- Conclusion

II.4. Processes vs Threads

Features	Process	Thread
Memory	Isolated	Shared
Creation Overhead	High	Low
Communication	Requires IPC	Direct vis shared memory
Context switching	Slower	Faster
Resource usage	More	Less
Termination	Slower	Faster

Outline

- What is a Process?
- What is a Thread?
- Processes vs Threads
- **Implementation of Processes and Threads**
- Conclusion

II.5. Implementation of Processes and Threads

II.5.1. Multiprocessing

It is a programming technique where multiple **independent processes** are created. Each process has its own dedicated memory space.

Characteristics:

- **Separate memory:** Processes do not share memory by default.
- **Heavyweight:** Creating and destroying processes is slower and consumes more resources than threads.
- **True parallelism:** Since each process has its own interpreter and memory space the operating system can schedule them to run simultaneously on multiple CPU cores.
- **Best for:** CPU-bound tasks, where the goal is to speed up computation by dividing the work across multiple cores.

II.5.2.Threading

It is a programming technique where multiple threads of execution exist **within a single process**. All threads share the **same memory space and resources of their parent process.**

Characteristics:

- **Shared memory:** All threads can access and modify the same global variables and data structures.
- **Lightweight:** Creating and destroying threads is faster and uses fewer resources than creating processes.
- **Concurrency, not always Parallelism:** Threads are often used for I/O-bound tasks While one thread is waiting, others can run.
- **Best For:** I/O-bound tasks, responsive user interfaces and tasks that require frequent, fast communication between concurrent units.

II.7. Conclusion

This chapter explored the foundational concepts of processes and threads, which are the building blocks of multitasking and concurrency in operating systems.

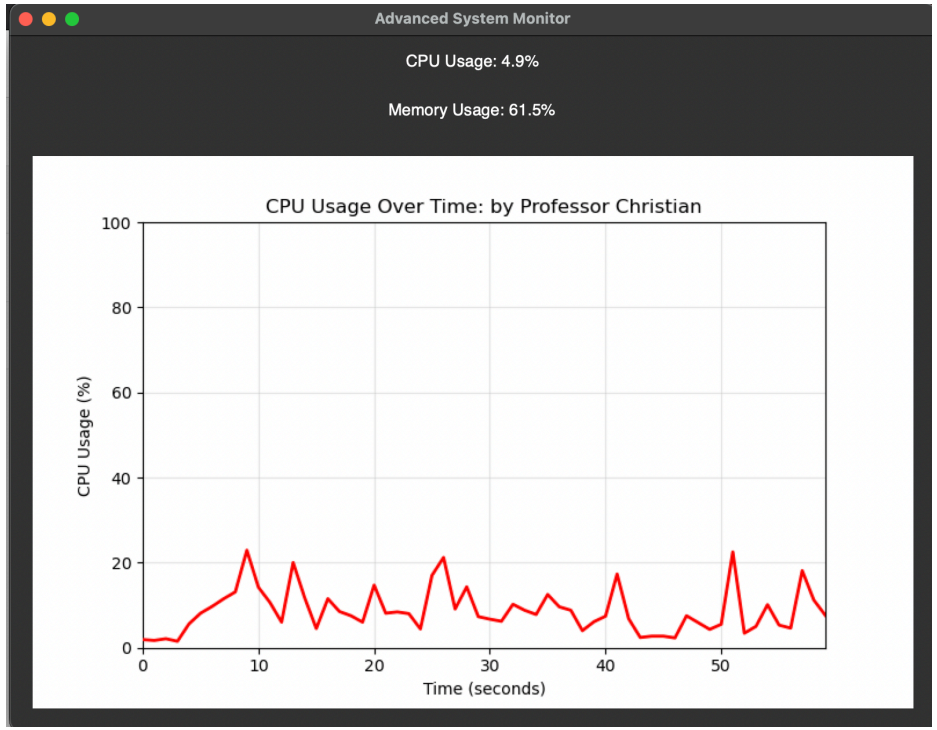
These basic but fundamental concepts help understand how an OS manages these entities. This knowledge actually appears to be crucial for designing efficient, responsive, and scalable application software that can be used in various platforms.

Lab

Objective: Create a dashboard to monitor CPU and memory usage in real time.

Tools:

- psutil: System info,
- tkinter: GUI
- Matplotlib: Graph



```python
import tkinter as tk
import psutil
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from matplotlib.animation import FuncAnimation
from collections import deque

# Set up the data structure for the graph
max_data_points = 60  # Show the last 60 seconds
time_data = deque(maxlen=max_data_points)
cpu_data = deque(maxlen=max_data_points)

# Initialize the data deques with zeros
for i in range(max_data_points):
    time_data.append(i)
    cpu_data.append(0)

def update():
    # Update the text labels
    current_cpu = psutil.cpu_percent()
    cpu_label.config(text=f"CPU Usage: {current_cpu:.1f}%")
    mem_label.config(text=f"Memory Usage: {psutil.virtual_memory().percent:.1f}%")

    # Update the data for the graph
    cpu_data.append(current_cpu)

    # Schedule the next update
    root.after(1000, update)

def update_graph(frame):
    # This function is called by the matplotlib animation
    # Update the graph data
    line.set_data(time_data, cpu_data)

    # Adjust the x-axis to scroll with the data
    if len(time_data) >= max_data_points:
        ax.set_xlim(time_data[0], time_data[-1])

    # Ensure the y-axis always shows 0-100%
    ax.set_ylim(0, 100)

    return line,
```

```python
# Create the main window
root = tk.Tk()
root.title("Advanced System Monitor")
root.geometry("800x600")  # Make window larger to fit the graph

# Create text labels
cpu_label = tk.Label(root, font=('Helvetica', 14))
mem_label = tk.Label(root, font=('Helvetica', 14))
cpu_label.pack(padx=20, pady=10)
mem_label.pack(padx=20, pady=10)

# Create matplotlib figure and axis
fig, ax = plt.subplots(figsize=(8, 4))
ax.set_title('CPU Usage Over Time: by Professor Christian')
ax.set_xlabel('Time (seconds)')
ax.set_ylabel('CPU Usage (%)')
ax.grid(True, alpha=0.3)

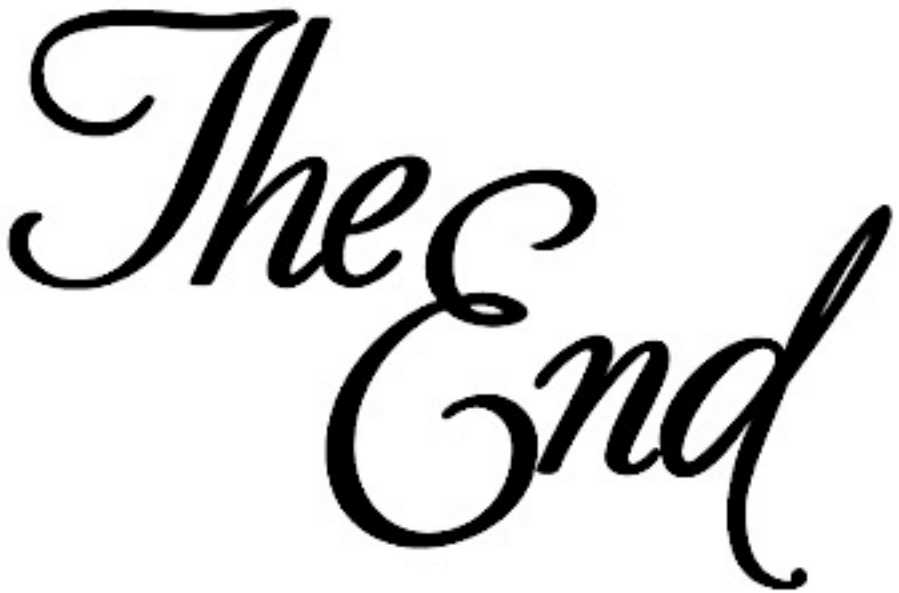
# Create the initial empty line
line, = ax.plot(time_data, cpu_data, 'r-', linewidth=2)
ax.set_ylim(0, 100)

# Embed the matplotlib figure in the Tkinter window
canvas = FigureCanvasTkAgg(fig, master=root)
canvas_widget = canvas.get_tk_widget()
canvas_widget.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

# Create the animation
ani = FuncAnimation(fig, update_graph, interval=1000, blit=True)

# Start the update cycle for the labels
update()

#print("Professor Christian")
# Start the GUI event loop
root.mainloop()
```


The End